SoloPrac: A Validity-Aware Multimodal Retrieval-Augmented Generation System for Clinical Records

Project Report submitted in the partial fulfilment
of
Bachelor of Technology
in
Computer Engineering

by

Ranveer Singh Thakur (C031)
Nihal Korgaonkar (C051)
Rohan Bhoge (C053)
Dev Patel (C054)

Under the supervision of

Dr. Dhirendra Mishra
(Professor, Department of Computer Engineering, MPSTME)

SVKM's NMIMS University
(Deemed-to-be University)



MUKESH PATEL SCHOOL OF TECHNOLOGY MANAGEMENT & ENGINEERING (MPSTME)
Vile Parle (W), Mumbai-56

Academic Year 2026-27

Certificate

This is to certify that the project entitled **"SoloPrac: A Validity-Aware Multimodal Retrieval-Augmented Generation System for Clinical Records"** has been done by **Mr. Ranveer Singh Thakur, Mr. Nihal Korgaonkar, Mr. Rohan Bhoge and Mr. Dev Patel** under my guidance and supervision, and has been submitted in partial fulfilment of the degree of Bachelor of Technology in Computer Engineering of MPSTME, SVKM's NMIMS (Deemed-to-be University), Mumbai, India.

Student Name	**Roll No.**
Ranveer Singh Thakur	C031
Nihal Korgaonkar	C051
Rohan Bhoge	C053
Dev Patel	C054

Project Mentor (Name and Signature)
(Internal Guide)

Examiner (Name and Signature)

Date: _____

Place: Mumbai

Head of Department (Name and Signature)

Acknowledgement

Dr. Dhirendra Mishra was our faculty mentor. Every week we brought them whatever we had, and the questions that came back changed the engineering as much as the research. Those reviews made us question our own assumptions. Several results in this report are here only because someone in a review asked something none of us had considered, which sent us back to measure it.

Thanks are also due to the Department of Computer Engineering at MPSTME, SVKM's NMIMS, for lab space and for the time to work on this over the year. A good part of what we did sits on top of other people's papers too; the authors we cite in Chapter 2 did that groundwork.

Name	**Roll No.**	**SAP ID**
Ranveer Singh Thakur	C031	70322100174
Nihal Korgaonkar	C051	70322100071
Rohan Bhoge	C053	70322100057
Dev Patel	C054	70322100150

Abstract

A lab reports a serum creatinine of 1.1 mg/dL, re-runs the sample a few days later and sends 1.4; in clinics this happens constantly. Both numbers are now in the chart. Only one is right. Systems that overwrite the first number lose the audit trail. Systems that keep both usually don't mark which one was withdrawn, and a retrieval-augmented answering system sitting on top will quote 1.1, citation and all.

We built SoloPrac to fix this at the level where it starts, which is storage. The events table only takes inserts; a correction is a new event that names the event it replaces, and whether a record is current, superseded or historical is worked out again every time it is read. An event counts as superseded exactly when some other event points at it. Since there is no status column, there is nothing that can fall out of step with the history.

For measurement we wrote TAP-QA, a synthetic benchmark. The first seed alone has 4,500 longitudinal records spread over 277 patients. Each seed asks 200 corrected and 400 historical queries. Only retrieval is scored, because generation is non-deterministic and we wanted numbers other people could reproduce.

Our retriever's stale-answer rate was 0.0 % on both seeds, at 97.8 % historical accuracy. Two results went against us and we report them too. We swept recency-decay weighting over 35 settings and historical accuracy stayed at zero for every one, so decay is a prior and nothing more. Also, a plain graph filter, which uses the same supersession edges and applies no weights, did just as well as our full scoring rule, so this report credits the relation itself. We had meant to present our weighting as the contribution. It isn't what produces the numbers.

What we hand in is a working web app, usable from login all the way to a downloaded PDF. Doctors and patients log into separate portals, history can be viewed as of any date, four types of document can be issued, and search covers text and images. If a photographed paper report can't be read with enough confidence, the app refuses to commit it. Each citation shows its own validity state, and the running cost is zero. The data, though, is entirely synthetic and we never ran a study with clinicians. Read the numbers as a workshop-scale result, since no clinic has tested them.

Table of Contents

Certificate i

Acknowledgement ii

Abstract iii

List of Figures vii

List of Tables viii

Abbreviations ix

1 Introduction 1

1.1 Background of the project topic 1
1.2 Motivation and scope of the report 1
1.3 Problem statement 2
1.4 Salient contribution 2
1.5 Organization of report 3
1.6 Summary 3

2 Literature Survey 4

2.1 Temporal data: the vocabulary the design adopts 4
2.2 Temporal information retrieval: why ranking is insufficient 4
2.3 Medical retrieval: the encoder and the benchmark landscape 5
2.4 Clinical interchange standards already store the relation 5
2.5 Temporal memory for language-model agents and versioned retrieval . . 5
2.6 Stale evidence in retrieval-augmented generation 6
2.7 Clinical benchmarks adjacent to the problem 6
2.8 Privacy and de-identification 6
2.9 Statistical treatment 6
2.10 Gap identified 7
2.11 Summary 7

3 Methodology and Implementation 8

3.1 Design principle: validity is derived, never stored . . . 8
3.2 Data model . . . 8
3.3 Three-valued validity . . . 10
3.4 Retrieval formulation . . . 10
3.5 Answer pipeline and honest degradation . . . 12
3.6 Multimodal retrieval and paper capture . . . 12
3.7 Standards adhered to . . . 14
3.8 Hardware description . . . 14
3.9 Software description . . . 15
3.10 Benchmark construction . . . 15
3.11 Summary of the chapter . . . 17

4 Results and Analysis **18**
4.1 Evaluation design . . . 18
4.2 Main results . . . 18
4.3 The recency-decay sweep: a negative result . . . 20
4.4 The graph-filter comparison: revising the attribution . . . 22
4.5 Statistical treatment . . . 23
4.6 The generation arm . . . 24
4.7 Verification . . . 24
4.8 Performance . . . 24
4.9 Threats to validity . . . 25
4.10 Summary of the chapter . . . 25

5 Advantages, Limitations and Applications **26**
5.1 Advantages . . . 26
5.2 Limitations . . . 26
5.3 Applications . . . 27
5.4 Summary . . . 27

6 Conclusion and Future Scope **28**
6.1 Conclusion . . . 28
6.2 Scope for future work . . . 28

References **30**

A Soft Code Flowcharts **33**
A.1 Capture flow . . . 33
A.2 Validity resolution algorithm . . . 33
A.3 Scoring rule . . . 34

B Data Sheets **35**

C List of Components **37**

D List of Papers Presented and Published **38**

List of Figures

Fig. 3.1 SoloPrac system architecture 9
Fig. 3.2 Database schema 11
Fig. 3.3 Validity of one record at two instants 12
Fig. 3.4 Flow of a chat request 13
Fig. 3.5 Issued prescription PDF 16
Fig. 3.6 Patient portal 16

Fig. 4.1 Staleness against historical accuracy, seed 20260601 20
Fig. 4.2 Corrected HbA1c in the running app 21
Fig. 4.3 Recency-decay sweep 22
Fig. 4.4 Correction step in the browser test suite 25

List of Tables

Table 2.1 Comparison of approaches to clinical record correction 7

Table 3.1 Failure modes and the behaviour each produces 14
Table 3.2 Hardware used 14
Table 3.3 Software components 15

Table 4.1 Benchmark composition for TAP-QA (seed 20260601) 18
Table 4.2 Retrieval performance by method (seed 20260601) 19
Table 4.3 Retrieval performance by method (seed 20260714) 19
Table 4.4 Recency-decay sweep results (seed 20260601); rows for $\lambda \in \{0.05, 0.1, 0.15, 0.3\}$ follow the same pattern and are left out . . 21
Table 4.5 Validity-aware retrieval against the graph filter 22
Table 4.6 Statistical comparisons against each baseline (seed 20260601) . 23
Table 4.7 Threats to validity and their status 25

Table B.1 Model specifications 35
Table B.2 Cache and encoding statistics 36

Table C.1 Software components of the deliverable 37

Table D.1 Manuscript under submission 38

Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
BM25	Best Matching 25 (probabilistic ranking function)
CI	Confidence Interval
DAG	Directed Acyclic Graph
DPDP	Digital Personal Data Protection Act, 2023 (India)
EHR	Electronic Health Record
FHIR	Fast Healthcare Interoperability Resources (HL7 standard)
GPU	Graphics Processing Unit
HL7	Health Level Seven (clinical interchange standards body)
LLM	Large Language Model
MRR	Mean Reciprocal Rank
nDCG	Normalised Discounted Cumulative Gain
OCR	Optical Character Recognition
PDF	Portable Document Format
PWA	Progressive Web Application
R@1	Recall at rank one
RAG	Retrieval-Augmented Generation
RRF	Reciprocal Rank Fusion
SAR	Stale Answer Rate
TAP-QA	Temporally Adversarial Patient Question Answering (the benchmark built in this work)

Chapter 1
Introduction

Introduction

First we describe the problem and why we went after it. A formal statement and our list of contributions come next. Chapter 2 goes through the literature on temporal data and retrieval that we built on and points out what is missing. Chapter 3 covers the design and how we implemented it. Chapter 4 has results, Chapter 5 the advantages, limitations and uses of the system, and Chapter 6 is the conclusion.

1.1 Background of the project topic

Our target user is a general practitioner in India working without partners, who sees perhaps fifteen patients in a day. Records there live in paper folders, in phone photos of lab reports, and now and then in whatever software the clinic once installed. When somebody fixes a transcription mistake, it tends to go wrong in one of two ways. The old value is overwritten (goodbye audit trail), or it is left in the file and nobody marks it as wrong. Time gets ignored as well. Ask for the current haemoglobin and you may well get last year's reading. And uncertainty stays out of sight: even a system that knows a value is stale might say nothing, so the clinician learns about it only by asking the right follow-up.

We do not think a better model would fix any of this, because the damage is done earlier, in how the record is stored. What shaped our project was realising that database researchers dealt with the same issue decades ago. A temporal database keeps two clocks apart: valid time, when a fact held in the real world, and transaction time, when the database found out about it [1, 2]. Under that discipline a correction is an insert (the old row stays put) and the state of the record on any past date is something you query for, so no stored copy is needed.

1.2 Motivation and scope of the report

Medicine already uses retrieval-augmented generation a great deal, and there are established benchmarks for it [3, 4]. All of them assume that once a fact is in the record it stays true, and under that assumption a benchmark cannot catch a system that answers with a value the clinician already threw out. Ranking by recency doesn't help. Recency hints at how relevant a passage probably is; it says nothing about whether the value is still correct, and the survey of temporal information retrieval by Campos and colleagues lands in the same place [5]. The original formulation of retrieval-augmented generation

was designed for knowledge that does not change over the period being asked about [6]. Nothing in it tells a passage that is still true apart from one that used to be true and was later replaced.

On purpose, the scope is small. SoloPrac deals with clinical supersession and nothing else: a clinician knew a value was wrong and corrected it. Temporal question answering over clinical text was around before us, so we are not claiming the first time-aware benchmark either. What we contribute is a benchmark for this one correction case, a retriever that passes it, and a shipped product in which that same retriever is the component under test.

1.3 Problem statement

We need a little notation. E is the set of clinical events; each event e has a valid time $v(e)$ and a recorded time $\tau(e)$, and a correction also names $s(e)$, the event it supersedes. Back to the creatinine from the abstract. Someone asks "what is the patient's creatinine?" in the present tense: 1.4 has to come first, and 1.1 must never be shown as current. If they ask about an instant t before the correction took effect, 1.1 is correct, since at t that is what the chart said. The present-tense requirement looks trivial on paper. It isn't, since the 1.1 event itself carries no mark of having been withdrawn: the only evidence is a second event, the 1.4 one, pointing back at it. A ranker that scores events one at a time never sees that pointer.

1.4 Salient contribution

1. **Derived validity.** No status column exists anywhere in the events table, and rows are only ever added. Validity is recomputed from the supersession relation at read time, so a stale status flag can't happen (there is no flag to go stale).
2. **TAP-QA**, a benchmark built around the clinical correction case. We score it at retrieval level, which keeps results reproducible even though generation is not deterministic.
3. **A negative result, kept in:** over a 35-point recency-decay sweep, historical accuracy never once rose above zero. Decay works as a prior. It is not a mechanism.
4. **Credit where the data puts it.** Our full scoring does no better than a baseline that simply filters on the supersession graph and weights nothing, so the relation is what we claim.
5. **A working product that costs nothing to run**, with an automated check behind each claim in this report that anyone can run again.

1.5 Organization of report

If you only want the results, skip to Chapter 4; it has every number, the negative ones too. Before that, Chapter 2 goes through prior work (temporal databases, time-aware and ordinary retrieval, clinical data standards, the recent agent-memory systems, medical RAG evaluation) and ends with the gap, and Chapter 3 explains how we built the system, from data model to software stack. Chapter 5 is advantages, limitations and applications. Chapter 6 wraps up with future work, and the appendices hold the flowcharts, data sheets, component list and publication record.

1.6 Summary

Corrections in a chart get appended, never applied in place, and retrieval that scores one entry at a time is blind to which entry was withdrawn. This project models that relation explicitly and measures what it buys. As the later chapters show, it lets a system avoid stale answers without giving up on questions about the past.

Chapter 2

Literature Survey

Introduction to the survey

This chapter goes over the earlier work our design depends on and then pins down the gap. We organised it around one question: when a clinical record has been corrected, what should a retriever give back? Some of the papers we cite cut against our approach. We kept them in, because most of the evidence for the design comes from seeing where existing methods break.

2.1 Temporal data: the vocabulary the design adopts

Snodgrass and Ahn put temporal databases on a formal footing and split valid time off from transaction time [1]. The terms people still use come from the consensus glossary by Jensen and colleagues [2]. In that glossary each fact has both kinds of time attached, and a fact that gets corrected stays in the database but stops being current after some point.

That split decided our data model. The events table has `valid_time` and `recorded_at` as two separate columns, and a correction goes in as an insert. To answer "what did we believe at date X?" we filter on `recorded_at` and derive validity again. Nobody has to keep historical status rows consistent.

2.2 Temporal information retrieval: why ranking is insufficient

A survey by Campos and colleagues covers temporal IR, from time-aware ranking to queries whose intent depends on time [5]. The useful point for us is that recency behaves like a prior. A newer document is more likely to be relevant. That says nothing about an older document being false, and a constraint would have to say exactly that.

The lexical family of rankers, BM25 included, comes from the probabilistic relevance framework of Robertson and Zaragoza [7]. Dense passage retrieval comes from Karpukhin and colleagues [8], and the standard way of fusing rankers is the method by Cormack and colleagues [9]. TempRALM, by Gade and Jetcheva, gives a retrieval-augmented language model an extra term for temporal proximity, which favours documents dated near the query [10]. Between them these give us the baselines tested in Chapter 4. None of them can see supersession, since it links two records and each method scores one record at a time.

We checked this with an experiment. Over a 35-point sweep of decay weights and half-lives, not one setting pushed historical accuracy above zero; Table 4.4 has the num-

bers.

2.3 Medical retrieval: the encoder and the benchmark landscape

MedCPT was contrastively pretrained on a large set of PubMed search logs, and it gives a query encoder and an article encoder sharing one 768-dimensional space [11]. It is our text encoder. Feasibility was part of the reason: it runs on our own machine and a query costs nothing.

MIRAGE, from Xiong and colleagues, and RGB, from Chen and colleagues, show that evaluating RAG is standard practice by now, in medicine and in general [3, 4]. They measure whether the right passage comes back from a record in which every statement stays true the whole time. In exactly that setting, a system that quotes a rejected value would still be scored as right. When documenting our data we follow Gebru and colleagues [12]. Our provenance terms come from the W3C PROV-DM recommendation [13]; its *revision* and *invalidation* relations are the general version of the supersession edge we use.

2.4 Clinical interchange standards already store the relation

A relation of the form "this entry replaces that one" is nothing new. In HL7 FHIR R4 an `Observation` can carry a status like `corrected` or `entered-in-error`, with the revised predecessor recorded through a provenance entity [14]. In openEHR, each composition keeps all of its versions, and each version points back to the one before it [15]. Clinical data exchange stores this relation all the time. Pipelines that chunk and embed text throw it away just as often, and for clinical records nobody had measured what that costs. Our `supersedes_id` column is that same edge; we just carry it all the way through to retrieval.

2.5 Temporal memory for language-model agents and versioned retrieval

Outside healthcare, memory stores for language-model agents arrived at our mechanism on their own during 2025–2026. Zep keeps bitemporal graph edges, and an outdated edge is invalidated and left in place [16]. Engram adds an explicit supersession chain and filters by a point in time (as-of) first, with dense ranking after that [17]. MemStrata retires old values with a deterministic supersession rule and reports almost no stale facts for software and configuration data [18]. For versioned technical documents, VersionRAG turns version relationships into graph edges, and accuracy on questions that depend on the version goes from 58 % to 90 % [19]. We read all this as evidence that the design holds up. Still, every one of these works sits outside clinical records, and they do not treat stale answers and historical accuracy as two objectives pulling against each other.

Whether a recency prior could take the place of the edge is also left untested.

2.6 Stale evidence in retrieval-augmented generation

Two recent studies put numbers on how much outdated evidence hurts RAG. HoH finds that outdated passages damage answer accuracy more than irrelevant ones do, and they can mislead a model even when the current information sits right next to them [20]. The stale-document poisoning study used 317 verified knowledge reversals, medical ones among them; outdated retrieval flipped about a third of the answers the model would otherwise have got right, and a recency-aware re-ranker won back only a few points [21]. Those two papers are why Chapter 4 has a recency-decay sweep. A recency prior that can't fix staleness in general corpora probably won't fix it in a TAP-QA chart, where each correction comes 1, 3 or 14 days after the value it revises.

2.7 Clinical benchmarks adjacent to the problem

Some recent clinical benchmarks get close to our problem without quite covering it. MEDEC is about detecting and correcting errors inside a note [22]. It shows clinicians care about catching wrong entries, but what it asks is whether the note being read contains a mistake. ChronoMedKG and the ChronoTQA benchmark that comes with it test reasoning about when a disease began (its *onset*), using the literature [23]. That is a different time axis; ours is the date when what we knew about a recorded value changed. So our question differs from both. Once a clinician has retracted a correctly recorded value, does the system stop treating it as current? And if someone genuinely asks about the past, can it still find that value?

2.8 Privacy and de-identification

Answers are generated by an external language-model endpoint, and nothing that identifies a patient is allowed off the machine. According to a systematic review of de-identification methods for clinical text, surrogate substitution is the dominant approach, with leading systems reporting headline F1 above 98 %, though the review adds that they generalise poorly across clinical sub-domains [24]. We substitute typed placeholders and back them with a hard outbound assertion (Section 3.5). That F1 figure is not ours to claim. Our scope is narrower anyway, since a provider only ever sees the fields the retriever returns.

2.9 Statistical treatment

We evaluate the same questions under two rankers, so McNemar's paired proportions test fits [25]. We run it per seed and never pool, because pooling would hide the fact that one seed on its own is not significant. Queries from one patient are not independent

either, so beside the unclustered interval we also give a bootstrap clustered by patient.

2.10 Gap identified

Put together, the literature has temporal data modelling and principled ranking methods. It has interchange standards that store the supersession relation, and it has benchmarks for medical RAG. Nobody has joined these at the point a clinician actually cares about. In the work we reviewed, no benchmark checks whether a system avoids returning a value that a clinician explicitly corrected, and what sets such a value apart is that someone rejected it on purpose (it did not simply get old). It is a narrow gap. The approaches we looked at are compared in Table 2.1.

Table 2.1 Comparison of approaches to clinical record correction

Approach	**Handles a correction?**	**Keeps the audit trail?**	**Tells the clinician it is unsure?**	**Verdict**
Overwrite in place	No	No	No	Rejected: the record is lost
Archive or soft-delete flag	Partly	Weakly	No	Rejected: the flag can drift out of agreement with history
Recency decay [5]	No	No	No	Rejected as a mechanism; retained only as a baseline
LLM instructed to prefer recent facts	No	No	Unreliably	Rejected: non-deterministic and unevaluable
RAG with an as-of filter	Only for past questions	No	No	Partial: correct about history, silent about the present
Bitemporal ranking [1, 2]	Partly	Weakly	No	Baseline: sound, but expresses no explicit relation
Graph filter, no weighting	Yes	Yes	Yes	Kept as the key baseline, and competitive
SoloPrac (this work)	Yes, by construction	Yes	Yes, per citation	Adopted

2.11 Summary

We took two things away from the survey. One is that the correction case really has not been addressed. The relation already exists in clinical standards and in newer agent-memory systems, yet its effect on clinical retrieval had never been measured. The other was less pleasant for us. The weighting scheme we had planned to claim as our contribution turned out not to be what carries the result, and that shaped how we designed the evaluation in Chapter 4.

Chapter 3

Methodology and Implementation

Introduction

We start with the one decision that everything else depends on, and then go through the architecture, data model, retrieval, the answer pipeline and its failure handling, the standards we followed, and the hardware and software we ran it on.

3.1 Design principle: validity is derived, never stored

A clinical event has no status column. It counts as superseded if, and only if, some other event lists it in `supersedes_id`. In pseudo-notation:

```
write:   v2.supersedes_id = v1.id        (a correction is an INSERT)
read:    is v1 superseded?  <=>  exists e : e.supersedes_id = v1.id
```

The first payoff is that a correction can't damage history: corrections only append, and no code path ever edits or removes an event. A status column also can't lie for the simple reason that there isn't one, and so a whole class of bug disappears. History, finally, turns into a query; we never keep a stored copy of it.

There is a price. Since validity is derived at read time, the timeline and as-of endpoints have real work to do on every request. With tens of events per patient the cost is negligible (median 31.5 ms, Section 4.8). At millions of events it would not be, and we would need a materialised view along with some way of invalidating it. We designed for the scale we state and no further.

3.2 Data model

The schema (Fig. 3.2) is six tables. Three of them, accounts, doctors and patients, are only identity and profile data; the interesting one is events. Documents and notifications cover issued paperwork and the inbox. Validity lives in a view called `event_validity`, so nobody writes their own chain walk in the frontend or a report script and gets it subtly wrong. An event row has `valid_time` (true for the patient when?), `recorded_at` (typed in when?) and `supersedes_id`, a self-reference. We keep a SHA-256 of each event's canonical JSON as tamper evidence.

Four invariants are checked by constraints inside the database. We kept them out of application code on purpose, because checks written there only hold until somebody adds a second write path:

1. `supersedes_id` has to point at an event of the same patient (nobody corrects some-

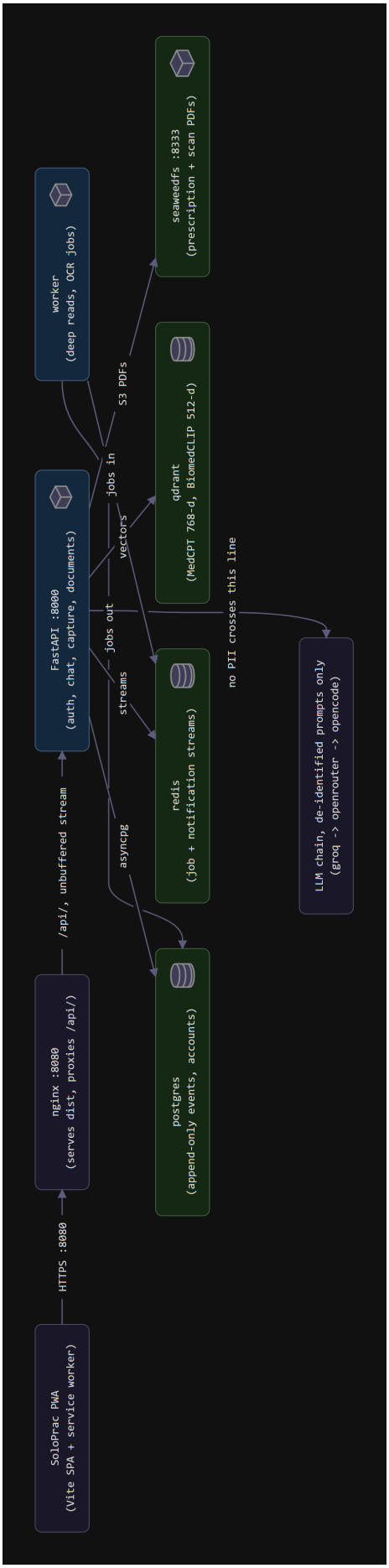


Figure 3.1 SoloPrac architecture as a block diagram: seven Docker services, with only de-identified text going out to the external language-model chain

one else's chart);
2. a correction's valid time is never before the valid time of the event it corrects;
3. nothing gets recorded before it happened, so `recorded_at` is never earlier than `valid_time`;
4. the correction graph has no cycles (A corrects B corrects A is rejected).

3.3 Three-valued validity

We use three validity values: *current*, *superseded* and *historical*, the last one meaning current as of some requested instant. Why a third? An event that got superseded today was still the right, current value back in March. If "correct back then" and "wrong now" were one state, the as-of view would give false answers. Formally,

$$\text{state}(e,t) = \begin{cases} \text{superseded} & \exists\, c :\ c \text{ corrects } e \ \wedge\ v(c) \le t \\ \text{current} & \text{otherwise,} \end{cases} \tag{3.1}$$

which means a correction knocks out its predecessor from the correction's own valid time onwards. Asking about an earlier instant still returns what was current then (Fig. 3.3). Appendix A lists the algorithm behind Eq. 3.1.

3.4 Retrieval formulation

Each candidate event gets a score that adds a semantic similarity to a validity weight taken from the supersession graph:

$$\text{score}(q,e) = \alpha\, \text{sem}(q,e) + (1-\alpha)\, w(e, \text{intent}), \qquad \alpha = 0.70, \tag{3.2}$$

where sem is cosine similarity under MedCPT, falling back to a lexical score when needed, and w takes one of a few fixed values: 1.00 if the event was current at the instant asked about, 0.85 if it is current and the question has no date in it, and 0.15 if it has been superseded. An event dated after the requested instant is thrown out completely, since a summary of the past should never quote a future result. Superseded events stay in, just lower down, because we wanted the generator to see that a correction happened. Ties between equally valid events go to whichever valid time is closest to the asked instant (the newest one does not automatically win).

The temporal intent comes from parsing the query. A present-tense question gets the current view, and one that names a past instant, say "as of 60 days ago", gets the as-of view. So one endpoint answers both kinds correctly and the caller never has to pick a mode.

One implementation detail deserves a note, since it gave us a fake result before we caught it. Tied candidates now get the average of their ranks under the method's own

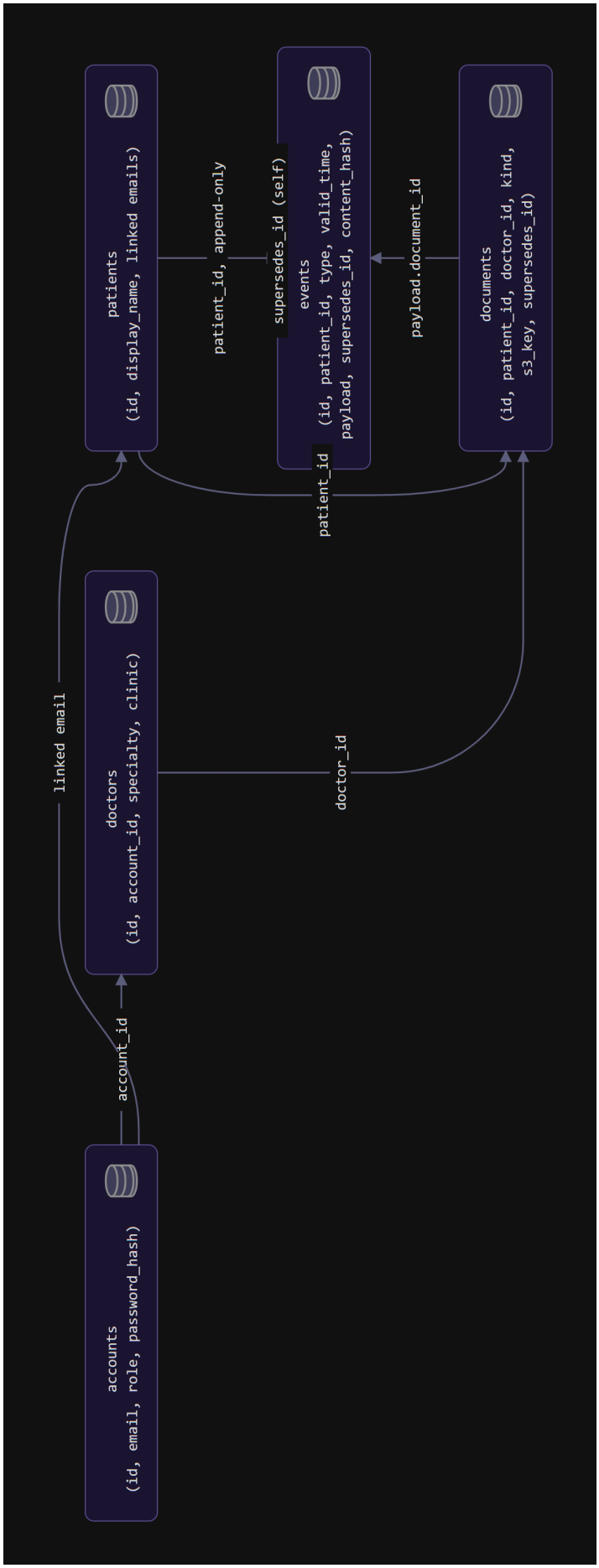


Figure 3.2 Database schema. Events are append-only and point to each other through `supersedes_id`; validity is a derived view

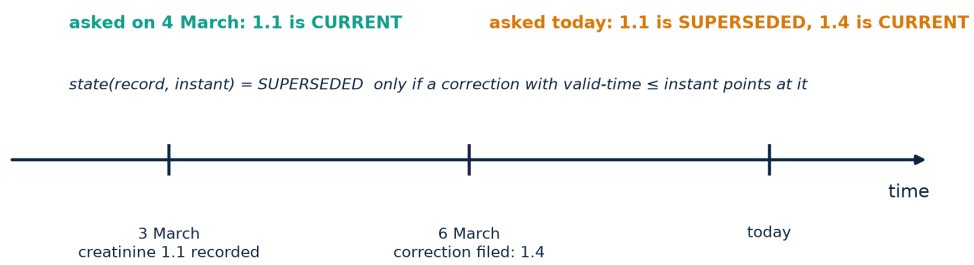


Figure 3.3 The same creatinine record is current on 4 March and superseded today: validity depends on the record and on the instant asked about

scores. In an earlier version the ordering of event identifiers broke ties instead, and that handed a token-overlap baseline a made-up historical accuracy of 99.8 per cent. We now have dedicated tie tests so this kind of bug can't creep back in.

3.5 Answer pipeline and honest degradation

Fig. 3.4 traces a chat request. We parse the temporal intent, retrieve with Eq. 3.2 and resolve validity for the instant asked about. The prompt is de-identified once, before any provider is contacted. Current facts go into the context first and superseded ones go in labelled. After the provider chain writes the answer, each claim is checked against its cited event twice: does the event support it, and does the event's validity state fit the claim's tense? ("HbA1c is 8.9" citing a superseded record fails the second check.) Citations come back tagged with their own validity. The model only ever sees surrogate IDs like R1 and R2, mapped back afterwards, so it cannot mangle a real identifier. Single-record questions skip the model and take about 6 ms.

Surrogate substitution is the dominant approach according to the review we cited [24], which is why we picked it, though the review's F1 numbers belong to other systems and we don't claim them. Our version is plain rules for names, dates, long digit runs, Aadhaar-shaped numbers, e-mail addresses and Indian phone numbers. It gets away with that because a provider only sees the few fields the retriever returns. And the outbound assertion blocks. A failed check means no request.

We care most about the bottom two rows. No model? You get an extractive answer and a visible notice, never an invented one. Lose the vector index and you have lost a cache: answers get a bit worse, the data is untouched.

3.6 Multimodal retrieval and paper capture

MedCPT (its query and article towers, 768-d) encodes the text records. Images go through BiomedCLIP instead (512-d), and both end up in the same Qdrant collection as named vectors. A chest radiograph, for example, becomes a dated event that you can

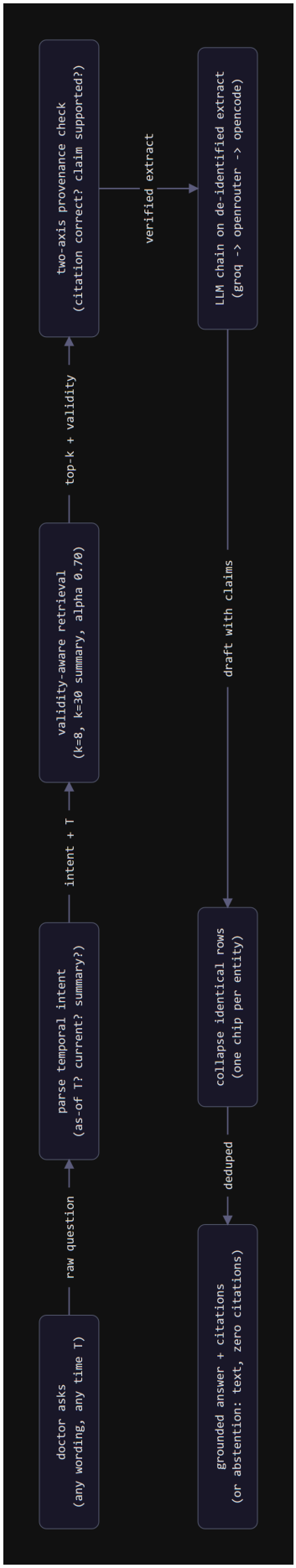


Figure 3.4 How a chat request flows through validity-aware retrieval, then de-identification and generation, and finally the provenance check

Table 3.1 Failure modes and the behaviour each produces

Failure	System behaviour
Primary model slow	12-second first-byte hedge, then the circuit breaker opens and the next provider is tried
Primary returns 4xx (not 429)	Raises a no-fall-back error, since retrying the same bad request elsewhere makes no sense
Primary rate limited (429)	Breaker opens for 15 minutes; the independent free fallback provider is used
No model available at all	Extractive fallback answers from retrieval alone; the interface says the model was unavailable
Vector index missing or cold	Lexical fallback; the search response is marked `degraded: true`; `POST /reindex` rebuilds the index from the event log

find by typing a description of it. The guard that keeps each patient's data separate is a filter clause inside the vector query itself. Paper reports arrive as phone photos. Before RapidOCR reads one (on the CPU), it has to get past a check on blur, contrast and resolution. Anything read with less than 0.75 confidence is not written to the record at all; the document is parked with `committed = false` and a person reviews it. For scans beyond OCR, a doctor can opt in to MedGemma 4B as a reading aid; its output also stays uncommitted until the doctor confirms it.

3.7 Standards adhered to

Our supersession semantics come from the `corrected` and `entered-in-error` values of `Observation.status` in HL7 FHIR R4, together with the revision relation in FHIR `Provenance` [14]. Versioning follows openEHR change control [15], and the provenance vocabulary is taken from the W3C PROV data model [13]. Every timestamp is in UTC (ISO 8601, timezone included). For object storage we use the S3 API, and issued documents are PDFs (ISO 32000). When designing the de-identification gate we worked from what India's Digital Personal Data Protection Act, 2023 asks for.

3.8 Hardware description

Table 3.2 Hardware used

Item	Specification
Processor / RAM	12th Gen Intel Core i5-12450H, 15.7 GB usable of 16 GB
GPU	NVIDIA GeForce RTX 3050 Laptop GPU, 4 GB
Storage	512 GB NVMe SSD; model weights bind-mounted read-only from a separate volume

The GPU only speeds up bulk embedding. At batch 16 the speed-up we measured was 2.9× and at batch 64 it was 12.9×, while a single query got 1.0× (no gain at all), since

a 4 GB laptop card is held back by launch latency. We run the vision language model on the CPU with memory mapping. Quantised, the model is about 2.4 GB and fits in system memory; keeping it off the GPU leaves accelerator memory for the retrieval path, which is the part we are actually evaluating. The cost on our laptop is 54.6 s per image and roughly 5.4 GB of resident memory. Far too slow for every upload, so a doctor has to switch the vision path on per request, and it never fires as an automatic fallback.

3.9 Software description

Table 3.3 Software components

Layer	Component
Frontend	React 18, TypeScript, Vite; installable progressive web application
Backend	FastAPI (Python 3.12), async SQLAlchemy
Worker	Separate process for asynchronous jobs (notifications, document commits, deep reads)
Primary store	PostgreSQL 16
Vector store	Qdrant 1.12 (768-dim text, 512-dim image, named vectors)
Pub/sub and cache	Redis 7
Object store	SeaweedFS (S3 API)
Reverse proxy	nginx
Orchestration	Docker Compose (7 services)
Text encoder	MedCPT query + article encoders
Image encoder	BiomedCLIP (PubMedBERT + ViT-B/16), loaded offline
OCR	RapidOCR (ONNX Runtime, CPU)
Vision LLM	MedGemma 4B (GGUF), opt-in, CPU with memory mapping
PDF rendering	WeasyPrint (HTML and CSS paged media)
Answer generation	Free-tier language-model chain: OpenCode Zen, Groq, OpenRouter, with circuit breaker

One command brings up the whole system, seven services in all. A health endpoint says which services are up, whether a language model has been configured, and whether the embedding models have warmed up yet. Figure 3.5 is a prescription from the running app, rendered as a watermarked PDF, and Figure 3.6 is the patient's portal after it arrived.

3.10 Benchmark construction

We wrote the TAP-QA generator to produce synthetic patient histories and plant corrections in them. Queries come in four classes. Some ask about facts that never changed, some about corrected facts; historical ones ask about a value that was later superseded, and a few are contradiction queries, where a note disputes a recorded value. We also gave every patient distractor corrections on tests that no query asks about, because otherwise a method could score well by demoting every correction it sees. To stop corrections from being obvious, each one shifts the value by 12 to 55 % of that test's reference range and its date is 1, 3 or 14 days later than the original observation. The generator does not read

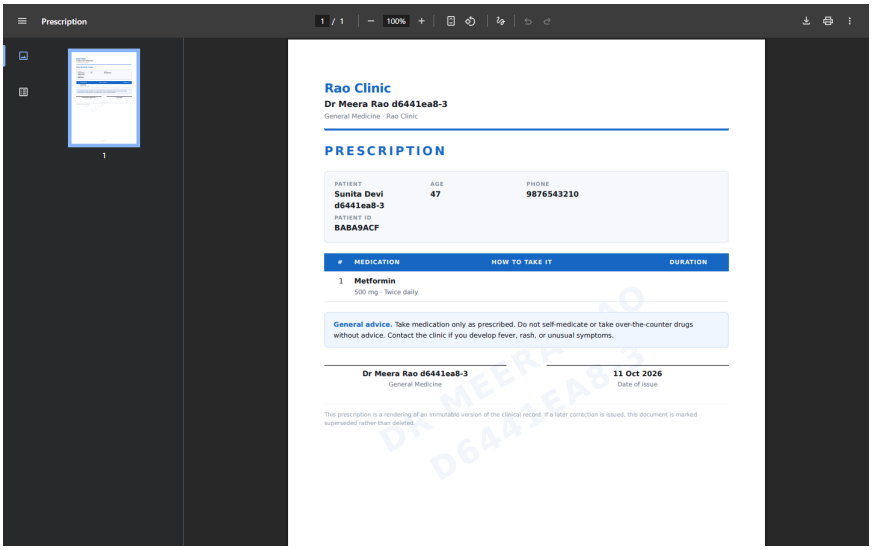


Figure 3.5 Implementation: a doctor-issued prescription as a watermarked PDF, with the doctor's registration, issue date and provenance footer

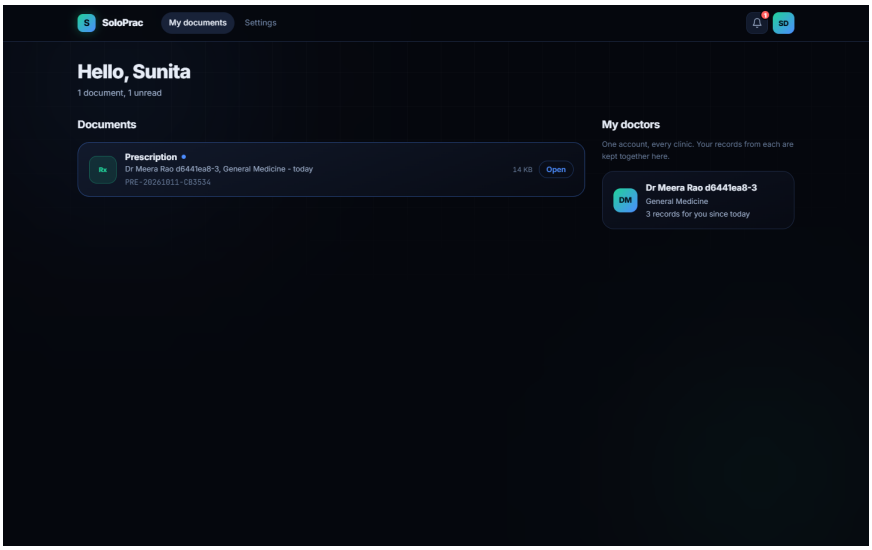


Figure 3.6 Implementation: the patient portal, showing the inbox of issued documents and the linked doctor

the system clock at all; same seed, same bytes. Following the datasheets idea [12] we documented the data, and none of it comes from real patients.

3.11 Summary of the chapter

Looking back over the chapter, nearly every decision follows from deriving validity at read time and never storing it. That one choice gave us the data model, the shape of the retrieval formula and the degradation behaviour. Running cost is zero on a laptop, the design sticks to clinical standards that already store supersession, and the app tells you when it isn't ready or something broke.

Chapter 4

Results and Analysis

Introduction

Every number in this chapter is read straight from the stored result files, never copied by hand, so the prose and the experiments can't disagree. Two of the findings below were bad news for our original plan, and they made us change what we claim as the contribution.

4.1 Evaluation design

We evaluate retrieval and keep the generated answer out of the main claim. Generation is non-deterministic, so the generation arm is reported separately, as a finding, in Section 4.6. The stale answer rate (SAR) counts corrected-value queries whose rank-one record had already been superseded by the time asked about. Historical accuracy counts as-of queries whose rank-one record was the valid one on that date. Corrected accuracy is joint: the top record has to be the gold record and must not be superseded. Unchanged accuracy is a control. A method that always answers with the newest record would look great on SAR and terrible here, and the control catches exactly that.

We ran two seeds independently. The first seed, 20260601, gives 277 patients with 4,500 records between them (972 are corrections) and 850 queries. The second, 20260714, gives 274 patients, 4,506 records including 1,001 corrections, and 852 queries. On each seed we run 45 configurations, namely the ten methods in Table 4.2 plus the 35 points of the recency-decay grid in Table 4.4. Once the embedding cache is warm, one full run finishes in 57 seconds. Two runs back to back produce identical bytes except for the timestamp and latency fields.

Table 4.1 Benchmark composition for TAP-QA (seed 20260601)

Query class	Count	What it tests
Unchanged	200	Accuracy on facts that were never corrected must not drop
Corrected	200	The corrected value has to beat the one it replaced
Historical	400	Return what was believed correct at a past instant
Contradiction	50	Surface a note that disputes a recorded value

4.2 Main results

Our method's stale answer rate is 0.0 per cent and its historical accuracy 97.8 per cent, meaning 391 of the 400 as-of queries came back right. Seed two gives exactly the same

Table 4.2 Retrieval performance by method (seed 20260601)

Method	SAR (%)	Hist. (%)	Corr. (%)	Unch. (%)	R@1
Random floor	17.0	9.2	4.5	7.5	0.074
Token overlap	100.0	0.0	0.0	97.5	0.229
Dense only, no temporal term	49.5	45.8	50.5	97.5	0.587
Reciprocal rank fusion, no temporal term	69.5	55.0	1.5	55.5	0.394
Valid-time only	0.0	0.0	100.0	99.0	0.521
Transaction-time only	21.5	21.2	78.5	99.0	0.554
Latest-N	0.0	0.0	10.5	5.0	0.039
Bitemporal, both axes, no graph	7.0	6.5	87.5	94.5	0.496
Graph filter, then dense	0.0	98.2	100.0	97.5	0.953
Ours: three-valued, graph-conditioned	**0.0**	**97.8**	**98.5**	**94.5**	**0.935**

Table 4.3 Retrieval performance by method (seed 20260714)

Method	SAR (%)	Hist. (%)	Corr. (%)	Unch. (%)	R@1
Random floor	22.5	7.2	5.5	5.4	0.068
Token overlap	100.0	0.0	0.0	98.0	0.232
Dense only, no temporal term	53.5	53.0	46.0	98.0	0.612
Reciprocal rank fusion, no temporal term	73.0	60.5	1.5	55.4	0.419
Valid-time only	0.0	0.0	99.5	99.0	0.522
Transaction-time only	24.0	23.2	75.5	99.0	0.568
Latest-N	0.0	0.0	11.5	5.9	0.045
Bitemporal, both axes, no graph	11.5	7.5	83.0	95.0	0.492
Graph filter, then dense	0.0	98.2	99.5	98.0	0.952
Ours: three-valued, graph-conditioned	**0.0**	**97.8**	**96.0**	**96.5**	**0.942**

pair. Zero stale answers out of two hundred corrected queries puts the 95 per cent upper bound on SAR at about 1.9 per cent. So, strictly, we can claim "below two per cent". The baselines all shifted from one seed to the other; SAR went up 4.5 points for bitemporal, 5.5 for random and 4.0 for dense-only. Our two headline figures stayed put.

Two of the baselines taught us something. Ranking on valid time alone gets zero stale answers and also zero historical accuracy. Our benchmark dates every correction later than the value it fixes, so this ranker happens to pick the correcting record for present-tense questions, while as-of questions are beyond it altogether. Latest-N shows the same degenerate pattern from the other side: good on staleness, close to chance on historical questions, the signature of always picking the newest record. We also tried the other dating convention, giving each correction the valid time of the original specimen. Valid-time ranking, which had been at 0 per cent stale, goes to 100; bitemporal goes from 7 to 70. Ours stays at 0, because it never compares timestamps and reads state straight off the graph. All 45 configurations are plotted against the two objectives in Figure 4.1.

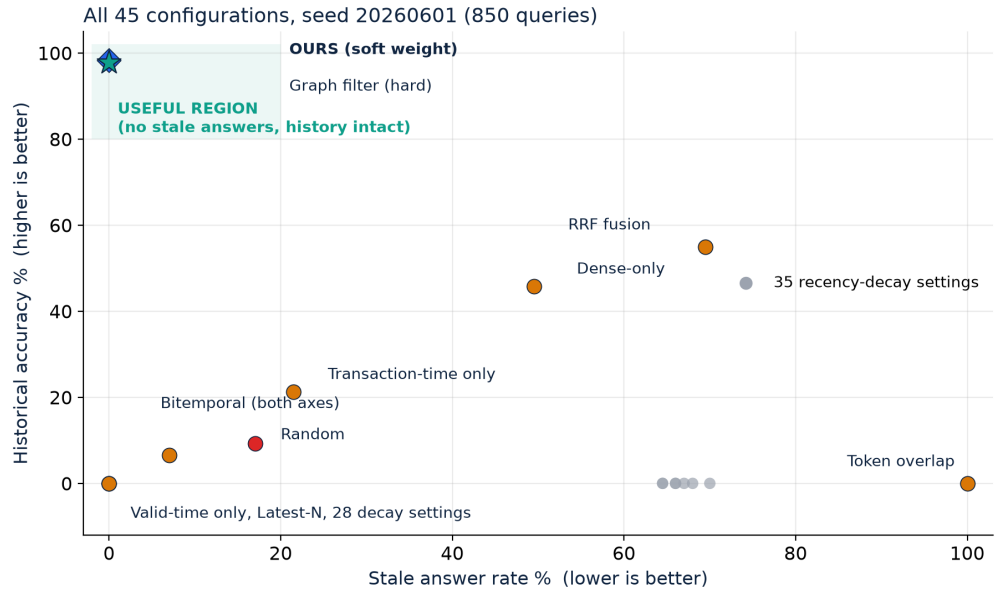


Figure 4.1 Seed 20260601, all 45 configurations, plotted by staleness against historical accuracy. Only the two graph-resolved rankers reach the upper left; the 35 recency-decay settings (grey) end up in two corners of the horizontal axis

4.3 The recency-decay sweep: a negative result

We crossed seven decay weights, $\lambda \in \{0.01, 0.03, 0.05, 0.1, 0.15, 0.3, 0.5\}$, with five half-lives, $\tau \in \{7, 30, 90, 180, 365\}$ days. That is 35 settings, and every one of them scored 0.0 per cent historical accuracy, on both seeds (Fig. 4.3). The cause is simple enough. Both versions of a corrected test have the same test name and nearly the same wording, so the similarity term scores them identically and the decay term alone de-

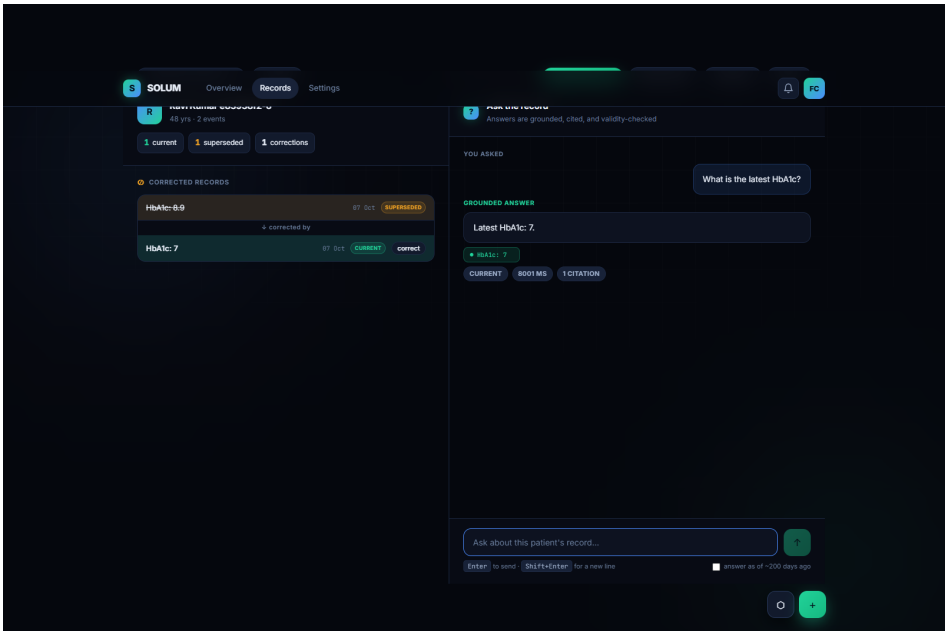


Figure 4.2 A corrected HbA1c in the running app: the superseded value stays, labelled, and the present-tense answer cites only the current one

Table 4.4 Recency-decay sweep results (seed 20260601); rows for $\lambda \in \{0.05, 0.1, 0.15, 0.3\}$ follow the same pattern and are left out

Decay weight λ	Half-life τ (days)	Historical accuracy (%)	Stale answer rate (%)
0.01	7	0.0	70.0
0.01	30	0.0	0.0
0.01	90	0.0	0.0
0.01	180	0.0	0.0
0.01	365	0.0	0.0
0.03	7	0.0	68.0
0.03	30	0.0	0.0
0.03	90	0.0	0.0
0.03	180	0.0	0.0
0.03	365	0.0	0.0
		…	
0.5	7	0.0	64.5
0.5	30	0.0	0.0
0.5	90	0.0	0.0
0.5	180	0.0	0.0
0.5	365	0.0	0.0

cides the order. The newer record is the right answer to a corrected question and, by construction, the wrong answer to every historical one. One dial cannot get both classes right. At $\tau = 7$ days something else kicks in too: over the age range of the records the decay term spans about twenty-five orders of magnitude, the two versions end up closer together than a float can resolve, and since our ranking is tie-aware, neither gets credit. That gives 64.5 to 71.5 per cent stale answers.

We kept this result in and limited the claim to what we tested, which is the weighting family in this sweep and nothing wider. In practice it means that a system needing guaranteed freshness won't get it from a decay prior, however carefully the prior is tuned.

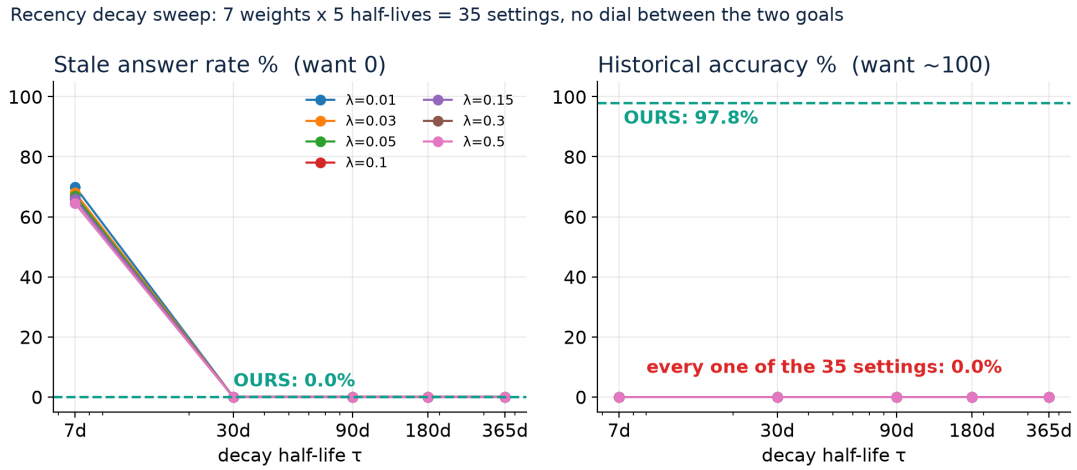


Figure 4.3 Recency-decay sweep over 35 settings, stale answer rate on the left and historical accuracy on the right. Every setting lands in one of two degenerate corners, and historical accuracy never rises above zero

4.4 The graph-filter comparison: revising the attribution

Table 4.5 Validity-aware retrieval against the graph filter

Seed	Method	SAR (%)	Historical (%)	Corrected (%)	R@1
20260601	Graph filter	0.0	98.2	100.0	0.953
20260601	Ours	0.0	97.8	98.5	0.935
20260714	Graph filter	0.0	98.2	99.5	0.952
20260714	Ours	0.0	97.8	96.0	0.942

The graph-filter baseline uses our supersession edges to remove what is invalid and then ranks the rest by similarity, without any validity weight. Seed for seed, its historical accuracy and its joint corrected score are as good as our full method's or a little better (McNemar $p = 0.25$ on seed 1, and $p = 0.016$ favouring the filter on seed 2). For the 200 unchanged queries it scores 97.5 and 98.0 per cent, and our soft rule 94.5 and 96.5, so our weighting actually costs two to three points there. We also varied the mixing weight

α, trying seven values from 0.40 up to 0.95. No headline number changed, because the spread of the validity weight is larger than any spread the similarity term produces.

This changes our claim, so we want to be upfront about it. When the project started, the plan was to present the validity-weighted ranking as the contribution. The data says otherwise. The supersession edge does the work, and the weighting is engineering hygiene: defensible when candidate sets are larger or embeddings noisier, but it is not what produces the result here. We still ship the demote-not-drop rule. With it, the generator still sees the correction in its context, and the system should degrade more gently if an edge is missing or arrives late. That second point is a design motivation; we have not measured it. Chapter 1 now states the attribution to match.

4.5 Statistical treatment

We ran one paired comparison per baseline, nine in all, using McNemar's exact test on the paired proportions [25] for each seed separately. For the difference in historical accuracy we also bootstrapped over patients (10,000 resamples). Seed 20260601's numbers are in Table 4.6. Against bitemporal, our historical accuracy is higher by +0.913. With patient clustering the 95 per cent interval is $[+0.875, +0.948]$. Ignoring clustering narrows it slightly, to $[+0.885, +0.940]$.

Table 4.6 Statistical comparisons against each baseline (seed 20260601)

Baseline	**McNemar p (safe answer)**	**Historical bootstrap 95 % CI**
Random floor	5.10×10^{-57}	$[+0.853, +0.915]$
Token overlap	9.96×10^{-60}	$[+0.960, +0.993]$
Dense-only (no temporal term)	7.97×10^{-27}	$[+0.458, +0.585]$
RRF fusion, no temporal term	7.97×10^{-59}	$[+0.357, +0.497]$
Valid-time only	0.25	$[+0.960, +0.993]$
Transaction-time only	4.62×10^{-10}	$[+0.708, +0.823]$
Latest-N	9.34×10^{-52}	$[+0.960, +0.993]$
Bitemporal, both axes, no graph	2.74×10^{-5}	$[+0.875, +0.948]$
Graph filter (as-of admissible, then dense)	0.25	$[-0.013, 0.000]$

We made the safe-answer outcome joint deliberately. A ranker can avoid stale answers entirely by declining to answer, so staleness on its own can be gamed (Latest-N: 0 per cent stale, 0 per cent historical). There are two caveats. First, on both seeds the bootstrap interval for (our method minus graph filter) includes zero. So on this data the two cannot be told apart, hence the attribution to the relation. Second, we applied no multiple-comparison correction across the nine comparisons.

4.6 The generation arm

Does better ranking change what the model actually says? We ran a paired generation experiment to find out. We took forty corrected queries and forty unchanged controls, and the same hosted model answered each one twice: once from the context our ranker surfaces, once from the dense-only ranker's context. Neither arm had any validity annotation. At $k = 1$, meaning one record of context, the validity-aware arm was never stale in 40 tries. The baseline was stale 18 times (45 per cent; McNemar $p = 8 \times 10^{-6}$). Was our arm just being cautious? The unchanged controls say no, since there the two arms got 37 and 39 of 40 right. Things change at $k = 2$ and $k = 5$, where we saw no stale answers from either arm at all. With two or more records in the context, both versions of the test end up side by side, and the model picks the newer one without being told to.

We report the null at $k \geq 2$ as measured. For this benchmark and this free-tier model, the ranker's guarantee counts at rank one; whether generation could gain from validity-aware context in other settings is left open. Many real consumers sit at rank one anyway, such as anything that truncates to a single passage or quotes one citation. Generation being non-deterministic is the other reason our main claim sits at the retrieval level.

4.7 Verification

An automated browser suite drives the shipped application through twelve checks of the behaviour the evaluation relies on. They cover the correction pair, the two as-of views, the difference between present-tense and historical answers, and the validity label on every citation. We also have an end-to-end script of sixteen steps that asserts on every server response. It starts by registering a doctor, making a patient and adding records; asked a present-tense question, the system says 8.9 %. Then the value is corrected to 6.8 and the same question returns 6.8 % (the answer never asserts 8.9); asking as of sixty days back returns 8.9 % again. The script issues a prescription and uploads an unreadable scan, which is refused and sent to review. Last, the patient signs up with the same e-mail, the existing records get linked automatically, and the inbox is read. Figure 4.4 is a capture taken by the browser suite itself.

4.8 Performance

Medians from the live stack follow. Timeline and retrieval calls come back in 31.5 ms. If a chat question is about one record, no model is called and the reply takes 35.1 ms; once the language model gets involved it is 2 to 3 s, and the worst we saw on the free tier was 18 s. Writing a record costs 257 ms, which includes embedding it, indexing it and updating the graph. OCR of a single page: 31.2 ms. Turning a prescription into a PDF: 214 ms. The embedding cache holds 3,672 entries and logged 33,814 hits against 3,678 misses, about a 90 per cent hit rate, and that brought a full run down from eight minutes

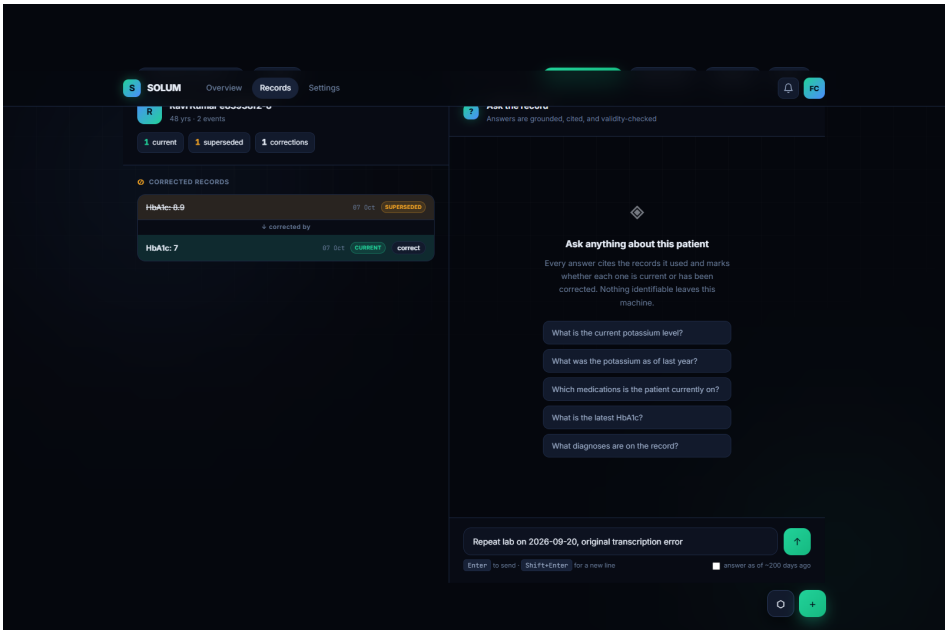


Figure 4.4 The correction step driven by the browser suite: superseded and current HbA1c values, with the correction reason typed into the console

to 57 seconds.

4.9 Threats to validity

Table 4.7 Threats to validity and their status

Threat	Status
All data synthetic; no clinical data available	Unresolved; this is the main limitation.
Every supersession edge present and correct; no measurement with missing or mis-pointed edges	Unresolved; probably the first thing a reviewer will raise.
No clinician study conducted	Unresolved.
Two seeds only	Better than one; more would be better.
No multiple-comparison correction across nine tests	Unresolved.
No cross-encoder or reranking baseline	Unresolved.
Bootstrap interval on ours minus graph filter contains zero	Reported; this is why we revised the attribution.
Corrections are laboratory values at depth one	Longer chains and other record types work and have unit tests, but are not in the benchmark.

4.10 Summary of the chapter

Both seeds show zero stale answers with historical accuracy still high, and the sweep shows recency decay cannot stand in for that. The graph-filter comparison moved the credit to the supersession relation, and we changed the claim to match. Generation benefits at rank one and shows a null at larger context sizes, both as measured.

Chapter 5

Advantages, Limitations and Applications

Introduction

This chapter lists strengths, then weaknesses, then possible uses. The weaknesses got as much space as the strengths, since anybody thinking of building on our work will mostly want to know where it is thin.

5.1 Advantages

- **Correct by construction.** Validity is derived from the correction relation. Since it is never stored, there is no way for the interface to think a record is current while the history says it was replaced.
- **The write is the audit trail.** Because corrections are appended, the value a clinician rejected is still sitting there; recovering it takes no extra machinery.
- **History is a first-class question.** Medico-legal reviews ask what was believed at a particular moment, and the as-of view answers exactly that.
- **Reproducible evaluation.** The headline claim is measured on retrieval, so anyone can check it. Give it one command and a seed and every number comes back.
- **Zero recurring cost.** All models run locally or on free tiers, and one command starts everything.
- **Failing in the open.** With the model down, it falls back to retrieval and tells you. If a scan can't be read reliably, it won't be written to the record.

5.2 Limitations

- **No clinical validation.** Every bit of evaluation data is synthetic and no clinician study took place. Nobody should use this system for clinical decisions; it has not been validated for that.
- **Supersession edges are taken as ground truth.** A real feed would have missing or mis-pointed edges, and we have not measured how the method copes when that happens.
- **No cross-encoder baseline.** We did not build a reranking stage, though it would probably change the picture. Chapter 4 compares only against first-order baselines.
- **No multiple-comparison correction** over the nine statistical comparisons.
- **No gain from generation at larger context sizes** for this particular benchmark and model.
- **Scale.** Deriving validity on read is fine at tens of events per patient. At millions it

would not be.

- **Images do not feed into generated prose yet.** Scans get indexed, dated and made searchable, but the generator gets OCR text and event payloads, nothing visual.
- **Vision is slow.** Roughly a minute per image on our target hardware. Hence opt-in.
- **No web push yet**; notifications only arrive in-app. Hindi retrieval is wired up, but we have not measured it.

5.3 Applications

- A one-doctor practice keeping its own records. Continuity of care there depends on knowing how values got corrected over time.
- Medico-legal review, where the question is what the doctor believed on a specific date.
- Teaching. Putting a corrected HbA1c next to the value it replaced is a quick way to show students why an audit trail is more than a log.
- Hospital systems that keep correction chains (HL7, FHIR) and surface them only as a stored status flag. There, our retrieval layer could read the edge directly.

5.4 Summary

Every advantage above comes from the same design decision. The limitations are what you would expect from a synthetic, two-seed study done at retrieval level: comparisons between methods hold up, but the raw numbers do not count as clinical evidence.

Chapter 6

Conclusion and Future Scope

Introduction

We close by looking back at what was done, drawing conclusions from the Chapter 4 results and their discussion, and listing what could come next. The conclusions are hedged, deliberately. Synthetic data and no clinical validation (both covered in Chapter 5) limit what we can say, and we try not to say more.

6.1 Conclusion

This project began with something that looked like a small record-keeping detail. It turned into a modelling question. After a clinician corrects a value, people will ask the system two different things: what is the value now, and what did we believe before? Store only one answer and sooner or later you're asked for the other. We derive validity from an explicit correction relation and never store it as a flag. That answers both questions, and a whole class of inconsistency simply can't arise, so nobody has to manage it.

Inside the scope we set, the evaluation supports the approach. On our benchmark (synthetic longitudinal records, with corrected and historical questions) the retriever had a 0.0 per cent stale answer rate on both seeds and got 97.8 per cent of historical questions right. Two other results qualify that. In 35 settings, recency decay never got historical accuracy off zero. Worse for our original plan, a baseline that applies the supersession edges but no weights matched the full method, so the contribution we now claim is the relation; that much the experiments support. Think of what a correction really is. The chart gains a newer entry, yes, but the important part is that the earlier entry has been taken back, and timestamps carry no trace of that. No amount of parameter tuning gets around it, because the missing piece is in the representation.

6.2 Scope for future work

- Noisy edges: drop or mis-point anywhere from 10 to 50 per cent of the edges, and see how badly things degrade when the feed is imperfect.
- Validation on real clinical records under clinician supervision, since this limitation restricts our claims more than any other. Radiology addenda in public critical-care corpora would give a real supersession graph, if we can get credentialed access.
- Adding a cross-encoder reranker. With harder candidates the weighting might finally earn its keep, or it might not.
- Running more seeds, and correcting for multiple comparisons across the significance

tests.

- Checking whether the generation null at larger context sizes comes from context dilution or from something about validity-aware context itself.
- Faster vision inference, so that automatic extraction becomes usable without an opt-in, and letting images inform the generated text.
- Erasure as a first-class operation. That means squaring the DPDP Act's right to erasure with an intact clinical audit trail.

References

[1] R. T. Snodgrass and I. Ahn, "Temporal databases," *IEEE Computer*, vol. 19, no. 9, pp. 35–42, Sep. 1986.

[2] C. S. Jensen, C. E. Dyreson, M. Böhlen, J. Clifford, *et al.*, "The consensus glossary of temporal database concepts—February 1998 version," in *Temporal Databases: Research and Practice*, O. Etzion, S. Jajodia, and S. Sripada, Eds., ser. LNCS, vol. 1399, Springer, Berlin, 1998, pp. 367–405.

[3] G. Xiong, Q. Jin, Z. Lu, and A. Zhang, "Benchmarking retrieval-augmented generation for medicine," in *Findings of the Association for Computational Linguistics: ACL 2024*, Bangkok, Aug. 2024, pp. 6233–6251.

[4] J. Chen, H. Lin, X. Han, and L. Sun, "Benchmarking large language models in retrieval-augmented generation," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 38, no. 16, 2024, pp. 17754–17762.

[5] R. Campos, G. Dias, A. M. Jorge, and A. Jatowt, "Survey of temporal information retrieval and related applications," *ACM Computing Surveys*, vol. 47, no. 2, article 15, pp. 1–41, 2014.

[6] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020, pp. 9459–9474.

[7] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[8] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proc. EMNLP*, Nov. 2020, pp. 6769–6781.

[9] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. ACM SIGIR*, Boston, Jul. 2009, pp. 758–759.

[10] A. Gade and J. Jetcheva, "It's about time: Incorporating temporality in retrieval augmented language models," *arXiv preprint arXiv:2401.13222*, Jan. 2024.

[11] Q. Jin, W. Kim, Q. Chen, D. C. Comeau, L. Yeganova, W. J. Wilbur, and Z. Lu, "MedCPT: Contrastive pre-trained transformers with large-scale PubMed search logs for zero-shot biomedical information retrieval," *Bioinformatics*, vol. 39, no. 11, article btad651, Nov. 2023.

[12] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III, and K. Crawford, "Datasheets for datasets," *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, Dec. 2021.

[13] L. Moreau and P. Missier, Eds., "PROV-DM: The PROV data model," W3C Recommendation, Apr. 2013. [Online]. Available: `https://www.w3.org/TR/prov-dm/`

[14] HL7 International, "FHIR Release 4: Resource Observation (element `status`) and Resource Provenance," 2019. [Online]. Available: `https://hl7.org/fhir/R4/observation.html`

[15] openEHR Foundation, "Common Information Model: Change control (versioning)," openEHR Reference Model specification. [Online]. Available: `https://specifications.openehr.org/releases/RM/latest/common.html`

[16] P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef, "Zep: A temporal knowledge graph architecture for agent memory," *arXiv preprint arXiv:2501.13956*, Jan. 2025.

[17] L. Wang, "Less context, more accuracy: A bi-temporal memory engine for LLM agents where a lean retrieved context beats the full history," *arXiv preprint arXiv:2606.09900*, Jun. 2026.

[18] N. Yadav, "Temporal validity in retrieval memory: Eliminating stale-fact errors for AI agents over evolving knowledge," *arXiv preprint arXiv:2606.26511*, Jun. 2026.

[19] D. Huwiler, K. Stockinger, and J. Fürst, "VersionRAG: Version-aware retrieval-augmented generation for evolving documents," *arXiv preprint arXiv:2510.08109*, Oct. 2025.

[20] J. Ouyang, T. Pan, M. Cheng, R. Yan, Y. Luo, J. Lin, and Q. Liu, "HoH: A dynamic benchmark for evaluating the impact of outdated information on retrieval-augmented generation," in *Proc. 63rd Annual Meeting of the ACL (Vol. 1: Long Papers)*, Vienna, Jul. 2025, pp. 6036–6063.

[21] M. S. Ahmed, L. G. Poech, and R. Röttger, "Stale-document poisoning: When outdated retrieval overrides correct model answers," *arXiv preprint arXiv:2609.31342*, Sep. 2026.

[22] A. Ben Abacha, W. Yim, Y. Fu, Z. Sun, M. Yetisgen, F. Xia, and T. Lin, "MEDEC: A benchmark for medical error detection and correction in clinical notes," in *Findings of the Association for Computational Linguistics: ACL 2025*, Vienna, Jul. 2025, pp. 22539–22550.

[23] M. S. Ahmed, F. Firoozbakht, L. G. Poech, J. Baumbach, and R. Röttger, "ChronoMedKG: A temporally-grounded biomedical knowledge graph and benchmark for clinical reasoning," *arXiv preprint arXiv:2605.22734*, May 2026.

[24] A. Kovačević, B. Bašaragin, N. Milošević, and G. Nenadić, "De-identification of clinical free text using natural language processing: A systematic review of current approaches," *Artificial Intelligence in Medicine*, vol. 151, article 102845, May 2024.

[25] Q. McNemar, “Note on the sampling error of the difference between correlated proportions or percentages,” *Psychometrika*, vol. 12, no. 2, pp. 153–157, Jun. 1947.

Appendix A

Soft Code Flowcharts

Introduction

This appendix has the flowcharts and the core algorithm. Every diagram started life as reladraw source text, which we render to vector and raster images; since the picture is generated from the text, the two can't drift apart. Figure 3.4 back in Chapter 3 already shows a chat request. Below are the capture flow and then the validity algorithm.

A.1 Capture flow

```
photo uploaded (JPEG/PNG/WebP/TIFF, <= 12 MB)
  -> quality gate: blur, contrast, resolution
  -> RapidOCR (CPU): text + per-line confidence
  -> field extraction: known test names, explicit unit map
       (unknown unit -> left blank, never guessed)
  -> confidence >= OCR_CONFIDENCE_THRESHOLD (0.75)?
       yes -> committed event (dated, indexed, citable)
       no  -> document held, committed = false, routed to human review
              (optional deep_read=true -> MedGemma draft, still uncommitted)
```

A.2 Validity resolution algorithm

This code comes from `backend/app/core/validity.py`. Every validity state in the app is decided in this one module, which has no side effects and 99 per cent unit-test coverage.

```
def state_as_of(self, event, instant):
    """SUPERSEDED at `instant` only if a correction whose own
    valid_time is at or before `instant` points at it."""
    if not self._children.get(event.event_id):
        return Validity.CURRENT
    for correction in self._children[event.event_id]:
        if correction.valid_time <= instant:
            return Validity.SUPERSEDED
    return Validity.CURRENT

def as_of_view(self, patient_id, instant):
    """Events that were true at `instant`, newest first."""
    return sorted(
        (e for e in self._by_id.values()
         if e.patient_id == patient_id
```

```
         and e.valid_time <= instant
         and self.state_as_of(e, instant) is Validity.CURRENT),
        key=lambda e: e.valid_time, reverse=True)
```

A.3 Scoring rule

```
for each candidate e of patient p, query q with parsed intent i:
    if i names an instant t and valid_time(e) > t:  drop e        (veto)
    w = 1.00 if state_as_of(e, t) is CURRENT          (as-of question)
      = 0.85 if state(e) is CURRENT and i has no date
      = 0.15 if SUPERSEDED                            (demoted, kept)
    score(e) = 0.70 * sem(q, e) + 0.30 * w
rank by score; break ties by |valid_time(e) - t|, not by recency
```

Appendix B

Data Sheets

Introduction

This appendix covers the models, what each is for and which of their quirks shaped the design, and ends with the benchmark's data sheet.

Table B.1 Model specifications

Model	Function	Specification	Notes
MedCPT query encoder	Embed the question	768-dimensional	Biomedical; runs locally; preloaded at boot
MedCPT article encoder	Embed record text	768-dimensional	Trained as a distinct tower; loaded on demand, saving about 430 MB resident
BiomedCLIP	Embed record images	512-dimensional	Loaded fully offline in 8.4 s; image and text vectors held under separate point identifiers
MedGemma 4B (vision)	Opt-in reading of scans OCR could not read	Quantised GGUF, about 2.4 GB on disk	CPU with memory mapping; 54.6 s per image, 5.4 GB resident; therefore opt-in and never an automatic fallback
RapidOCR	Read printed reports	ONNX Runtime, CPU	About 31 ms per page; abstains below 0.75 confidence

TAP-QA dataset card

TAP-QA is synthetic: longitudinal patient records into which we plant corrections. Seed 20260601 has 277 patients, 850 queries and 4,500 records, made up of 314 notes, 446 medications, 489 diagnoses and 3,251 laboratory results, with 972 corrections among them. Seed 20260714 has 274 patients, 852 queries and 4,506 records (1,001 corrections). It is meant for evaluating retrieval when records get corrected. There is no real patient data in it, and it should not be treated as a clinical dataset. Table 4.7 lists its known limitations.

Table B.2 Cache and encoding statistics

Quantity	**Value**
Embedding cache entries	3,672
Cache hits	33,814
Cache misses	3,678
Hit rate	90.2 per cent
Forward batches	1,004
Peak forward batch	18

Appendix C

List of Components

Introduction

SoloPrac has no hardware of its own, so by "components" we mean (as the guideline does) the software subsystems that make up the deliverable. The table below lists them.

Table C.1 Software components of the deliverable

Component	Technology	Responsibility
Web client	React 18, TypeScript, Vite	Doctor and patient portals, timeline, chat, document issuance, photo capture
API service	FastAPI	Routes for authentication, patients, events, timeline, chat, capture, documents, semantic search, notifications
Validity engine	Python	Supersession graph and three-valued validity derivation
Retriever	Python, MedCPT, Qdrant	Validity-aware ranking with lexical fallback
Answer pipeline	Python, free-tier LLM chain	De-identification, grounded prompt, two-axis provenance verification, citations
Document service	Python, WeasyPrint	Prescription, certificate, invoice and weekly report rendering
Worker	Python, Redis streams	Asynchronous jobs and notification fan-out
Stores	PostgreSQL, Qdrant, SeaweedFS, Redis	Records, vectors, issued artefacts, cache
Benchmark	Python (`eval/tapqa`)	TAP-QA generator, harness, baselines, statistics

We used no proprietary or purchased component. Each model runs either on our own machine or on a free tier, so the recurring cost is zero.

Appendix D

List of Papers Presented and Published

Introduction

We have written one manuscript from this work, and it is under submission. Its subject matter is TAP-QA, the retriever and our statistical analysis (roughly Chapters 2 to 4 here).

Table D.1 Manuscript under submission

Title	Venue	Status
"Corrected, Not Deleted: Supersession-Aware Retrieval over Longitudinal Clinical Records" (IEEE conference format, 5 pages)	Health-informatics and information-retrieval conferences with October 2026 deadlines (IEEE ICHI 2027, MIE 2027, HEALTHINF 2027, ECIR 2027 resource track, COMSNETS 2027 workshop)	Prepared; under submission

There is no reprint attached. At the time we submitted this report, nothing had been accepted yet.